

And this is the query for the ROWS clause. As we can see, things work exactly as in the previous example without the ORDER BY DESC option:

```

forumbd=> SELECT x,row_number() OVER w, dense_rank() OVER w,sum(x) OVER w
FROM (select generate_series(1,8) % 4 as x) V
WINDOW w AS (ORDER BY x desc ROWS BETWEEN 1 PRECEDING AND CURRENT ROW);
 x | row_number | dense_rank | sum
---+-----+-----+-----
 3 |          1 |          1 |    3
 3 |          2 |          1 |    6
 2 |          3 |          2 |    5
 2 |          4 |          2 |    4
 1 |          5 |          3 |    3
 1 |          6 |          3 |    2
 0 |          7 |          4 |    1
 0 |          8 |          4 |    0
(8 rows)

```

In this example, using the sum function, we can better understand the difference between the RANGE and ROWS options. As we can see, the RANGE option aggregates data by frame(RANGE) while the ROWS option aggregates data by rows. The main difference between the ROWS clause and the RANGE clause is that ROWS operates on individual rows, while RANGE operates on groups. That concludes our chapter on window functions.

Summary

In this chapter, we explored how to use window functions. We have seen that by using window functions, we can create more complex aggregates compared to those made with the GROUP BY statement, which we saw in *Chapter 5, Advanced Statements*. We learned how to use the ROW_NUMBER (), FIRST_VALUE (), LAST_VALUE (), RANK DENSE_RANK(), LAG (), LEAD (), CUME_DIST (), and NTILE () functions. We have also seen the difference between creating aggregates with the ROWS BETWEEN and RANGE BETWEEN clauses. You can use what you have learned in this chapter in data mining operations to make your work much easier.

For more information on window functions, you can consult the official documentation: <https://www.postgresql.org/docs/current/functions-window.html>.

In the next chapter, we will talk about server-side programming. We will look at how to create functions to be used on the server side and, if necessary, where to use window functions.

Verify your knowledge

- Consider these two queries:
 - `select category,count(*) from posts group by category order by category;`
 - `select category, count(*) over (partition by category) from posts order by category;`

Which of the two queries has a greater number of records?

The second query has a greater number of records.

See the *Using basic statement window functions* section for more details.

- Consider these two queries:
 - `select category,count(*) from posts group by category order by category;`
 - `select distinct category, count(*) over (partition by category) from posts order by category;`

Which of the 2 queries has a greater number of records?

The two queries have the same number of records.

See the *Using basic statement window functions* section for more details.

- Which of these two queries is semantically correct?
 - `select category,row_number() over w,title from posts WINDOW w as (partition by category order by title) order by category;`
 - `select category,row_number() over w,title from posts WINDOW w as (partition by category) order by category;`

The first one is semantically correct because the `row_number()` function depends on order by.

See the *The row number function* section for more details.

- Can we have the first value within a partition ?

Yes, we can, using the `first_value()` function. See the `FIRST_VALUE` section for more details.

- Can we do an incremental sum row by row in a table?

Yes, we can, using the clause `BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`

See the *Using advanced statement window functions* section for more details.

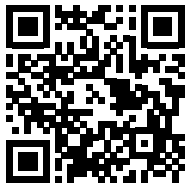
References

- PostgreSQL window functions official documentation: <https://www.postgresql.org/docs/current/functions-window.html>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



7

Server-Side Programming

In previous chapters, we learned how to execute SQL queries. We started by writing simple queries, then moved on to writing more complex queries; we learned how to use aggregates in the traditional way, and in *Chapter 5, Advanced Statements*, we talked about window functions, which are another way to write aggregates. In this chapter, we will add server-side programming to this list of skills. Server-side programming can be useful in many cases as it moves the programming logic from the client side to the database side. For example, we could use it to take a function that has been written many times at different points of the application program and move it inside the server so that it is written only once, meaning that in case of modification, we only have to modify one function. In this chapter, we will also look at how PostgreSQL can manage different server-side programming languages, and we will see that server-side programming can be very useful if you need to process a large amount of data that has been extracted from tables. We will address the fact that all the functions we will write can be called in any SQL statement. We will also see that in some cases, for certain types of functions, it is also possible to create indices on the functions.

Another feature of server-side programming is the chance to define customized data. In this chapter, we will look at some examples of this.

In simple terms, this chapter will discuss the following:

- Exploring data types
- Exploring functions and languages
- The NoSQL data type

Technical requirements

Before starting, remember to start the Docker container named `chapter_07`, as shown below:

```
$ bash run-pg-docker.sh chapter_07
postgres@learn_postgresql:~$ psql -U forum forumdb
```

Exploring data types

As users, we have already had the opportunity to experience the power and versatility of server-side functions—for example, in *Chapter 5, Advanced Statements*, we used a query similar to the following:

```
forumdb=> select * from categories where upper(title) like 'A%';
 pk | title |          description
----+-----+-----
  4 | A.I   | Machine Learning discussions
(1 row)
```

In this piece of code, the `upper` function is a server-side function; this function turns all the characters of a string into uppercase. In this chapter, we will acquire the knowledge to be able to write functions such as the `upper` function that we called in the preceding query.

In this section, we'll talk about data types. We will briefly mention the standard types managed by PostgreSQL and how to create new ones.

The concept of extensibility

What is extensibility? Extensibility is PostgreSQL's ability to extend its functionality and its data types. Extensibility is an extremely useful PostgreSQL feature because it enables us to have data types, functions, and functional indexes that are not present in the base system. In this chapter, we will cover extension at the data type level, as well as the addition of new functions.

Standard data types

In previous chapters, even if not explicitly obvious, we already used standard data types. This was when we learned how to use **Data Definition Language (DDL)** commands. However, we will now be looking more deeply into this topic. The following is a short list of the most used data types:

- Boolean type
- Numeric types
- Character types

- Date/time
- NoSQL data types: hstore, xml, json, and jsonb

For each data type, we will show an example operation followed by a brief explanation. For further information on the standard data types supported by PostgreSQL, please refer to the official documentation at <https://www.postgresql.org/docs/current/extend-type-system.html>.

Boolean data type

First, we will introduce the Boolean data type. PostgreSQL supports Boolean data types. The Boolean type (identified by BOOLEAN or BOOL), like all data types supported by PostgreSQL, can assume the NULL value. Therefore, a Boolean data type can take the NULL, FALSE, and TRUE values. The data type input function for the Boolean type accepts the following representations for the TRUE state:

State	true	yes	on	1
-------	------	-----	----	---

For the false state, we have the following:

State	false	no	off	0
-------	-------	----	-----	---

Let's look at some examples, starting with the users table:

1. Let's first display the contents of the users table:

```
forumbd=> select * from users;
pk | username | gecost | email
----+-----+-----+-----
 1 | luca_ferrari |      | luca@pgtraining.com
 2 | enrico_pirozzi |      | enrico@pgtraining.com
 3 | newuser |      | newuser@pgtraining.com
(3 rows)
```

2. Now let's add a Boolean data type to the users table:

```
forumbd=> alter table users add user_on_line boolean;
ALTER TABLE
```

3. Let's update some values:

```
forumbd=> update users set user_on_line = true where pk=1;
UPDATE 1
```

4. Now, if we want to search for all the records that have the `user_on_line` field set to `true`, we have to perform the following:

```

forumbd=> \x
Expanded display is on.
forumbd=> select * from users where user_on_line = true;
-[ RECORD 1 ]+-----
pk          | 1
username    | luca_ferrari
gecos       |
email       | luca@pgtraining.com
user_on_line | t

```

5. If we want the search for all the records that have the `user_on_line` field set to `NULL`, as we saw in *Chapter 4, Basic Statements*, we have to perform the following:

```

forumbd=> select * from users where user_on_line is NULL;
-[ RECORD 1 ]+-----
pk          | 2
username    | enrico_pirozzi
gecos       |
email       | enrico@pgtraing.com
user_on_line |
-[ RECORD 2 ]+-----
pk          | 3
username    | newuser
gecos       |
email       | newuser@pgtraining.com
user_on_line |

```

Thus, we have explored the Boolean data type.

Numeric data type

PostgreSQL supports several types of numeric data types; the most used ones are as follows:

- integer or `int4` (4-byte integer number).
- `bigint` or `int8` (8-byte integer number).
- real (4-byte variable precision, inexact with 6-decimal-digit precision).

- `double precision` (8-byte variable precision, inexact with 15-decimal-digit precision).
- `numeric (precision, scale)`, where the precision of a numeric is the total count of significant digits in the whole number, and the scale of a numeric is the count of decimal digits in the fractional part. For example, 5.827 has a precision of 4 and a scale of 3.

Now, we will look at some brief examples of each type in the upcoming sections.

Integer types

As we can see here, if we cast a number to an integer type such as `integer` or `bigint`, PostgreSQL will make a truncated value of the input number:

```
forumdb=> \x
Expanded display is off.
forumdb=> select 1.123456789::integer as my_field;
 my_field
-----
         1
(1 row)

forumdb=> select 1.123456789::int4 as my_field;
 my_field
-----
         1
(1 row)

forumdb=> select 1.123456789::bigint as my_field;
 my_field
-----
         1
(1 row)

forumdb=> select 1.123456789::int8 as my_field;
 my_field
-----
         1
(1 row)
```

Numbers with a fixed precision data type

In the following example, we'll see the same query that we have seen previously, but this time, we'll make a cast to real and to double precision:

```
forumdb=> select 1.123456789::real as my_field;
 my_field
-----
 1.1234568
(1 row)

forumdb=> select 1.123456789::double precision as my_field;
 my_field
-----
 1.123456789
(1 row)
```

As can be seen here, in the first query, the result was cut to the sixth digit; this happened because the real type has at least 6-decimal-digit precision.

Now suppose we want to perform the sum of the value 0.1 10 times. The correct result would be the number 1. Instead, if we execute:

```
forumdb=> select sum(0.1::real) from generate_series(1,10);
      sum
-----
 1.0000001
(1 row)
```

We get the value 1.0000001. This happens due to the intrinsic rounding error in the real data type, so it is not recommended to use the real data type in fields representing money. The correct way to make this sum is using the numeric data type.

Numbers with an arbitrary precision data type

In this last section about numeric data types, we'll make the same query that we saw earlier, but we'll make a cast to arbitrary precision:

```
forumdb=> select 1.123456789::numeric(10,1) as my_field;
 my_field
-----
```

```
1.1
(1 row)

forumdb=> select 1.123456789::numeric(10,5) as my_field;
 my_field
-----
 1.12346
(1 row)

forumdb=> select 1.123456789::numeric(10,9) as my_field;
 my_field
-----
1.123456789
(1 row)
```

As we can see from the examples shown here, we decide how many digits the scale should be. But what about if we perform something like the following?

```
forumdb=> select 1.123456789::numeric(10,11) as my_field;
ERROR:  numeric field overflow
DETAIL:  A field with precision 10, scale 11 must round to an absolute
value less than 10^-1.
```

The result is an error. This is because the data type was defined as a numeric type with a precision value equal to 10, so we can't have a scale parameter equal to or greater than the precision value.

Similarly, the next example will also produce an error:

```
forumdb=> select 1.123456789::numeric(10,10) as my_field;
ERROR:  numeric field overflow
DETAIL:  A field with precision 10, scale 10 must round to an absolute
value less than 1.
```

In the preceding example, the query generates an error because the scale was 10, meaning we should have 10 digits, but we have 11 digits in total:

Digits	1	2	3	4	5	6	7	8	9	10	11
	1	.	1	2	3	4	5	6	7	8	9